

UNLOCK

MODULE 22

From Q-Learning to Deep Q-Networks

Bridging classical reinforcement learning with deep neural networks to conquer complex, high-dimensional environments.

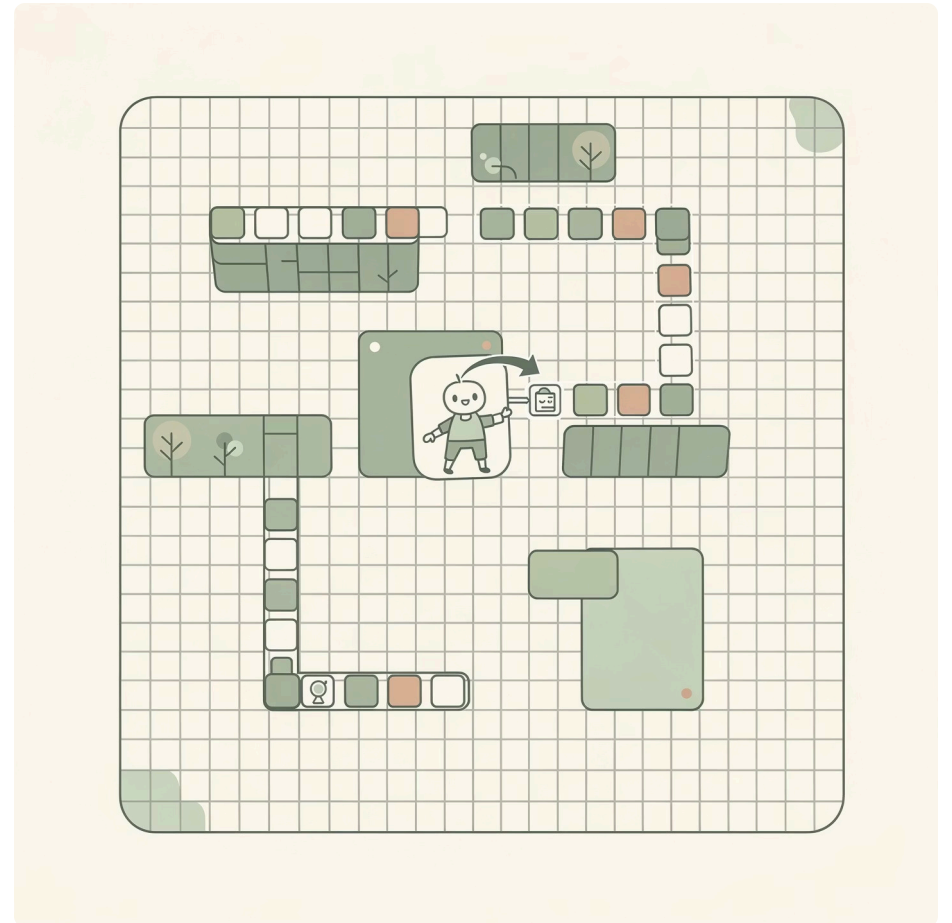


The Limits of Traditional Q-Learning

Traditional Q-learning stores value estimates in a **lookup table** — one entry per state-action pair. This works beautifully for small, discrete environments like grid worlds or simple board games.

But real-world problems quickly overwhelm this approach. Consider an Atari game screen: a single 210×160 RGB frame contains roughly **10 million possible pixel configurations**. Storing a Q-value for every possible screen and every possible action is computationally infeasible — the table would be astronomically large, and most entries would never be visited.

- ❑ The curse of dimensionality makes tabular Q-learning impractical for any environment with continuous or high-dimensional state spaces.



Why Tables Break Down

To appreciate the need for function approximation, let's look at what happens as the state space grows.

Grid World

A 4×4 grid with 4 actions yields just **64** Q-values. A table handles this effortlessly.

Board Game

Chess has ~1047 legal positions. Storing Q-values for every state-action pair is already impossible.

Video Game

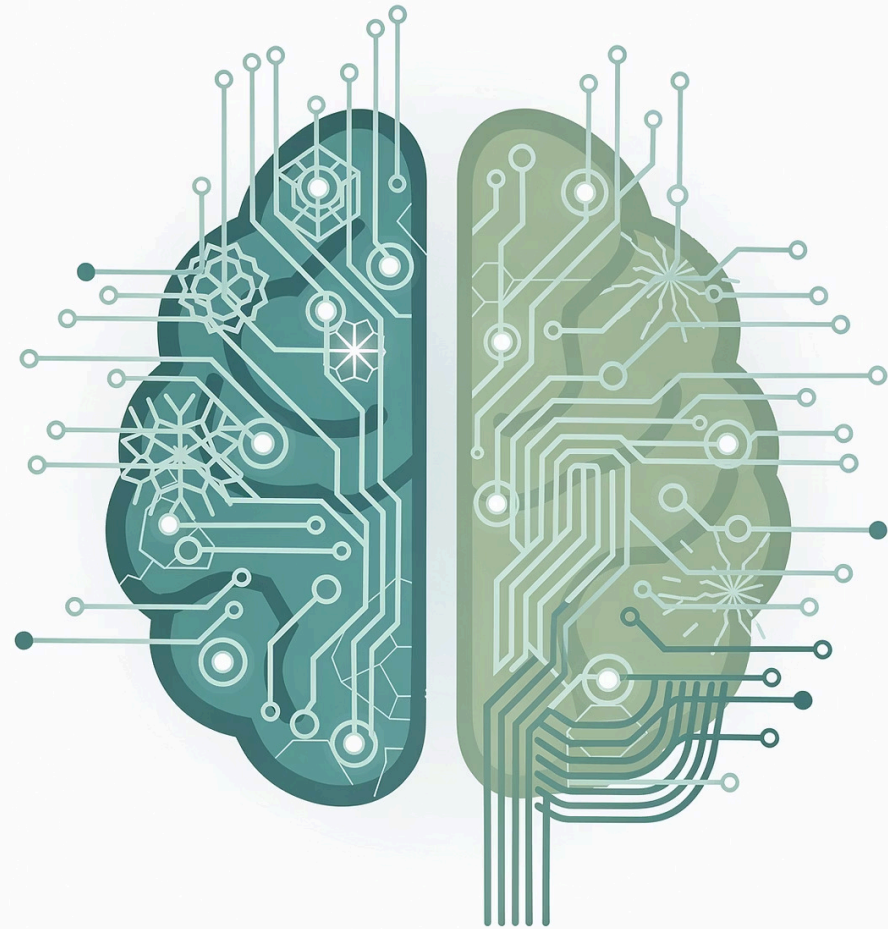
An Atari frame has ~107 dimensions. The agent must **generalize** — it can never visit every possible state.

The fundamental insight is clear: we need a way to **approximate** the Q-function rather than enumerate it. Enter deep neural networks.

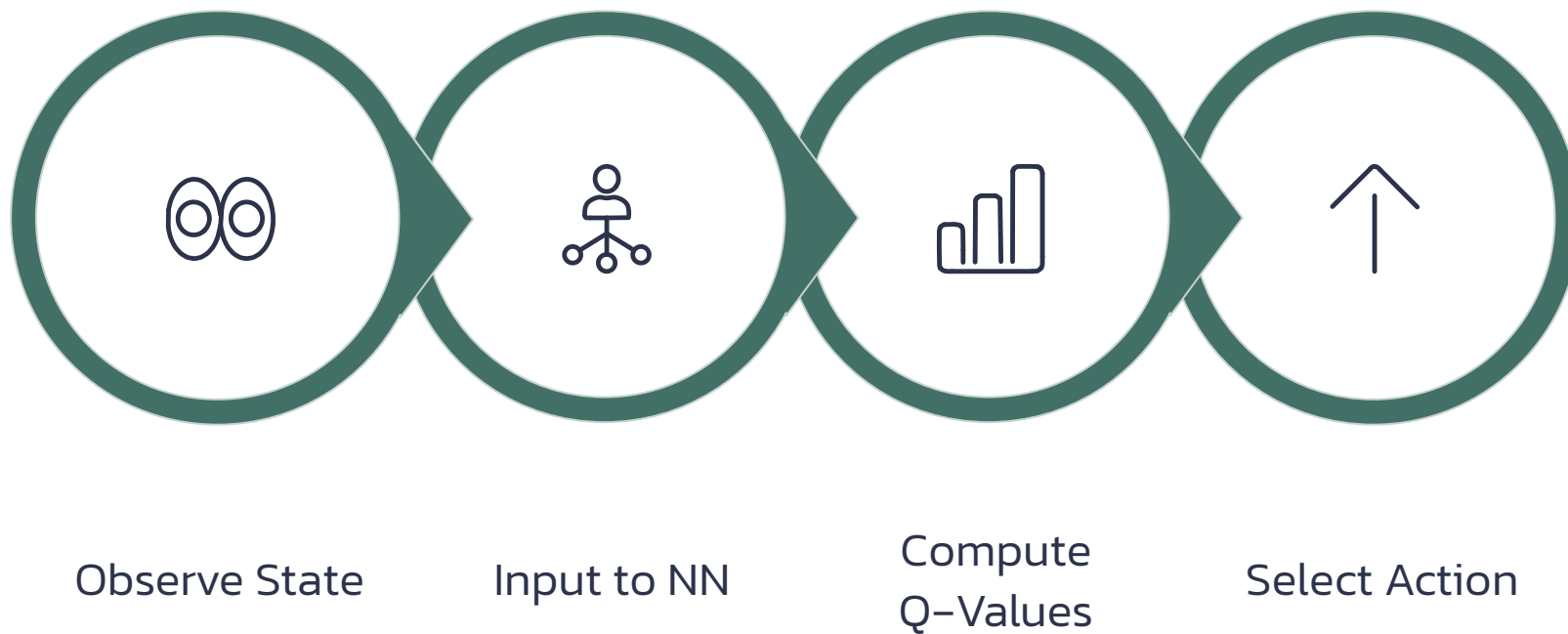
Enter Deep Q-Networks (DQN)

Deep Q-Networks solve the scalability crisis by replacing the Q-table with a deep neural network. Instead of looking up values in a table, the network *learns* a function that maps any state to Q-values for all possible actions.

This means the agent can generalize from past experiences and make intelligent decisions in states it has never encountered before — a dramatic leap in capability over tabular methods.



How DQN Works



The neural network takes the raw state (e.g., pixel frames, sensor readings) as input and produces a vector of Q-values — one for each possible action. The agent then selects the action with the highest predicted Q-value, following a policy that balances exploration and exploitation.

Key Innovation: Experience Replay

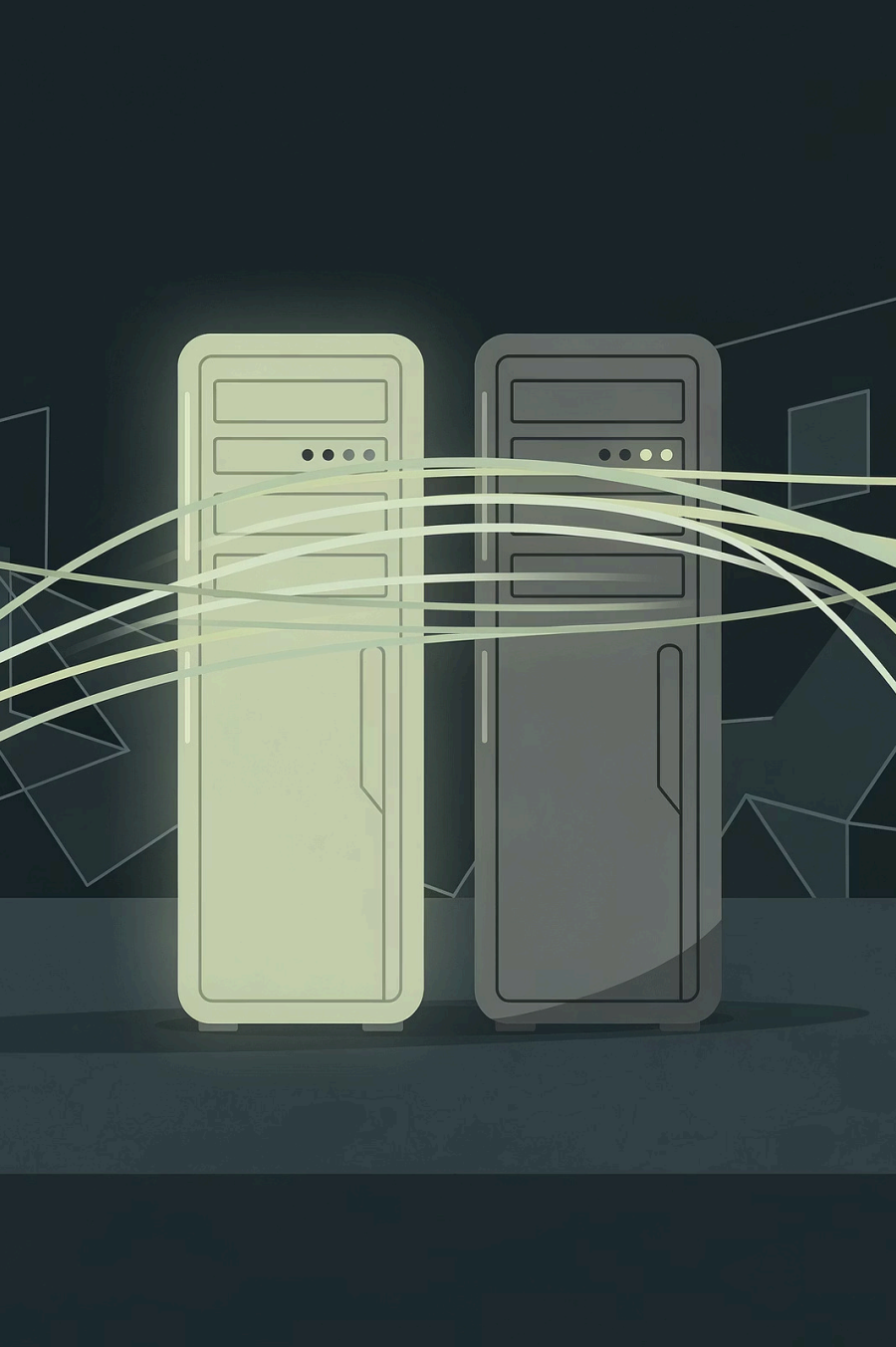
The Problem

Sequential training data is highly correlated — consecutive frames in a game are nearly identical. Training on correlated samples destabilizes neural networks and leads to poor convergence.

The Solution

DQN stores transitions (s, a, r, s') in a **replay buffer**. During training, the network samples random mini-batches from this buffer, breaking temporal correlations and reusing rare, valuable experiences multiple times.

This simple but powerful technique was one of the breakthroughs that made DQN work in practice, enabling the landmark 2015 DeepMind Atari results.



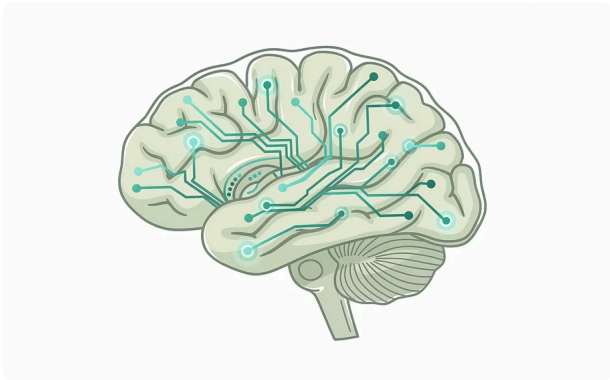
UNLOCK

CHAPTER 3

The DQN Architecture: Two Networks

A defining feature of DQN is its use of **two separate neural networks** — the policy network and the target network. This dual-network design is what prevents the notorious instability of naive deep RL training.

Policy Network vs. Target Network



Policy Network

The **main network** that is updated at every training step via gradient descent. It selects actions and generates the current Q-value predictions. Its weights change continuously as the agent learns from new batches of experience.



Target Network

A **frozen copy** of the policy network whose weights are held fixed for a set number of steps (e.g., every 1,000 updates). It provides stable Q-value targets for the loss function, preventing the "moving target" problem that causes divergence.

Why Two Networks Matter

Without a target network, the Q-value targets shift with every weight update — like trying to hit a moving target. This feedback loop causes oscillations and divergence.

1

Stability

Fixed target values decouple prediction from target computation, reducing harmful feedback loops during training.

2

Convergence

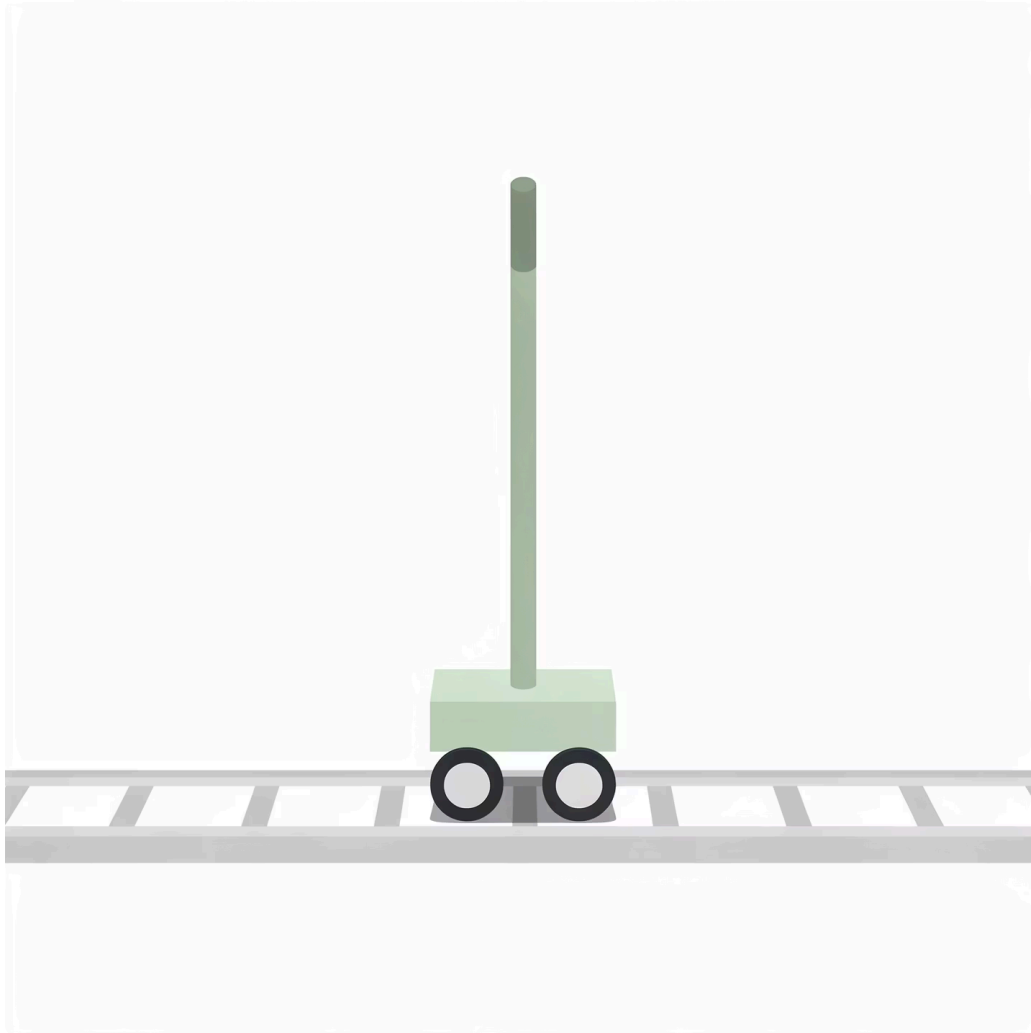
Periodic synchronization (copying policy weights $\rightarrow$ target) lets the target gradually improve without destabilizing learning.

3

Reproducibility

Stable training dynamics make results more reproducible and hyperparameter tuning more tractable across environments.

Application: The CartPole Environment



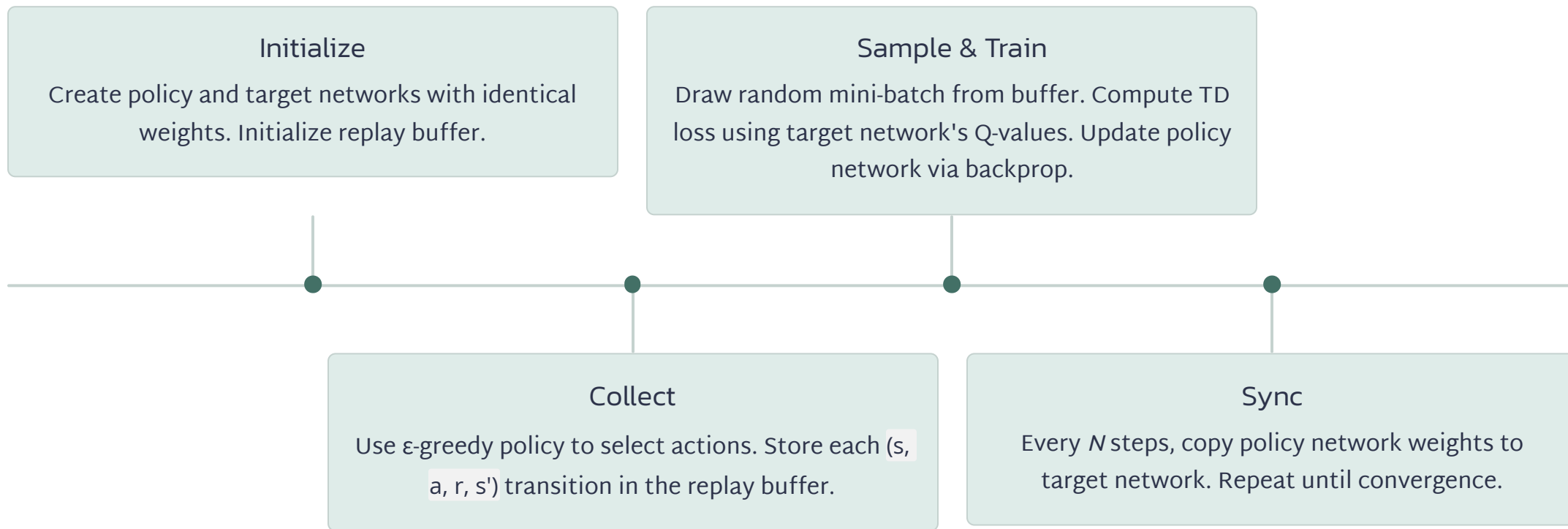
The **CartPole** problem is a classic benchmark from OpenAI Gym. The agent controls a cart that moves left or right along a frictionless track, and its goal is to keep a pole balanced upright for as long as possible.

State space: 4 continuous values — cart position, cart velocity, pole angle, and pole angular velocity.

Action space: 2 discrete actions — push left or push right.

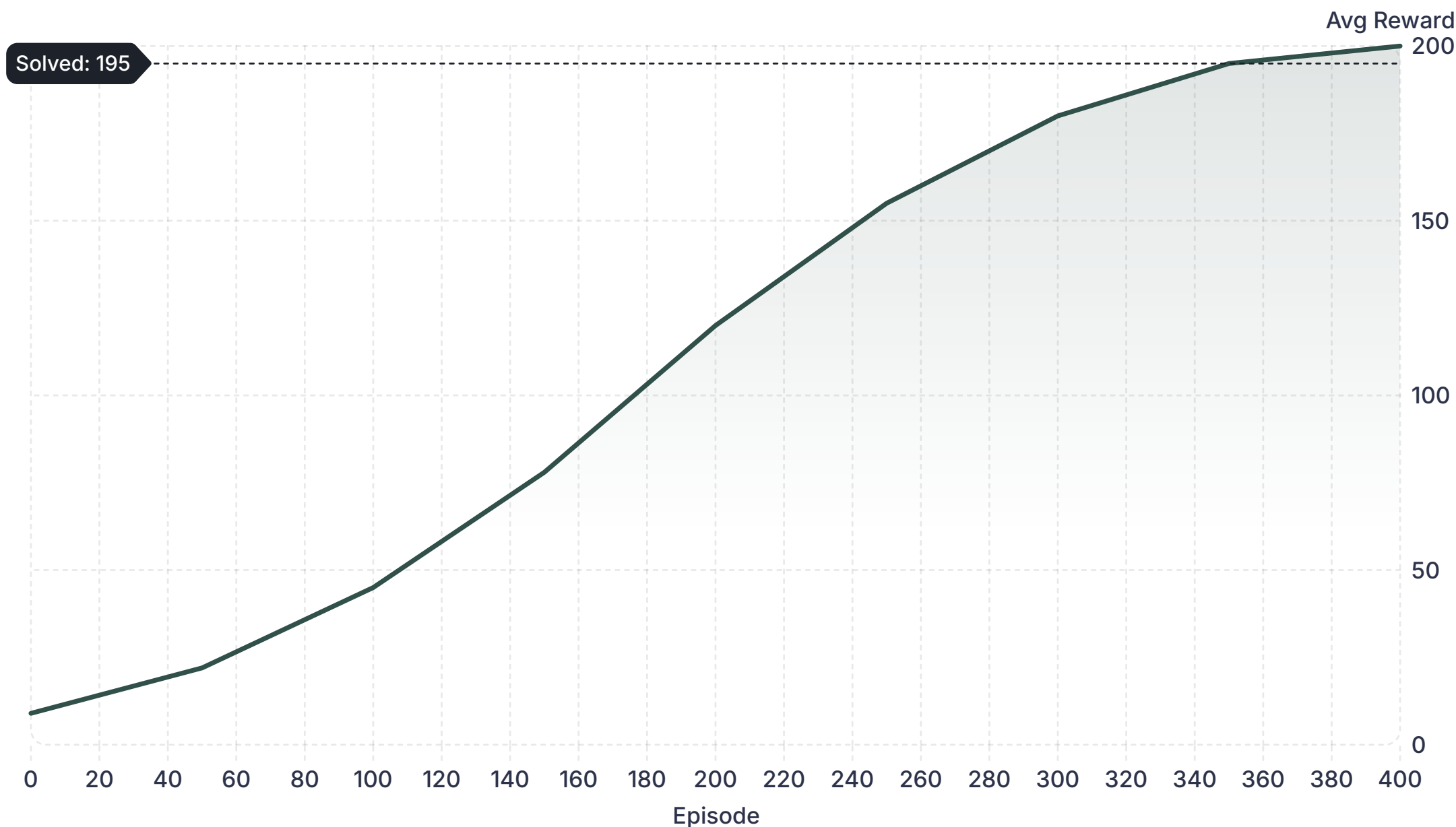
Reward: +1 for every timestep the pole remains upright. The episode ends when the pole falls beyond 15° or the cart moves off-screen.

DQN Training Loop for CartPole



With a well-tuned DQN, the CartPole agent typically solves the environment (achieving a reward of 200+ per episode) within a few hundred episodes of training — demonstrating how powerful neural function approximation can be even in a simple domain.

Expected Training Performance



A typical DQN training curve on CartPole shows rapid initial improvement followed by gradual convergence to the maximum reward of 200. The environment is considered "solved" when the agent achieves an average reward of 195+ over 100 consecutive episodes.



UNLOCK RECAP

Key Takeaways

1 Neural Function Approximation

DQN replaces the Q-table with a neural network, enabling reinforcement learning in high-dimensional and continuous state spaces where tabular methods fail.

2 Stability Through Architecture

The dual-network design (policy + target) combined with experience replay are the critical innovations that make deep Q-learning stable and convergent.

3 Practical Impact

From CartPole to Atari to robotics, DQN laid the foundation for modern deep reinforcement learning and remains a cornerstone technique in the field.